ITBA | EDUCACIÓN EJECUTIVA_

TRABAJO FINAL INTEGRADOR

Instrucciones y formato de entrega

Certificación Profesional en AI Agent Developer

Guía para la entrega, evaluación y defensa del proyecto final.

ENTREGA FINAL Domingo 8 de noviembre de 2026 - 23:59 h

Objetivo del TFI

Desarrollar una aplicación de Inteligencia Artificial funcional que resuelva un caso de uso real o realista e integre los principales conceptos trabajados durante la certificación.

PRIORIDAD Una solución acotada, estable y bien evaluada vale más que una arquitectura ambiciosa pero incompleta.

1. ¿Qué debe incluir el proyecto?

Los siguientes ocho componentes son obligatorios. Las etiquetas C9, C10, etc. indican las clases en las que se trabajan principalmente esos conceptos.

1	**Capacidad agéntica**	C9

- El sistema debe tomar decisiones sobre qué acción ejecutar para avanzar hacia un objetivo.
- Debe incluir al menos 2 tools, selección dinámica de acciones, validación de entradas/salidas y manejo de errores.
- No alcanza con ejecutar una secuencia fija de pasos: debe existir una decisión realizada por el agente.

2	**Skill reutilizable**	C10

- Incluir al menos una skill especializada vinculada con el dominio del caso de uso.
- Puede contener instrucciones, procedimientos, conocimiento específico, ejemplos, recursos o tools asociadas.

3	**Estado y memoria**	C10-C11

- Mantener información durante la ejecución y contar con algún mecanismo básico de persistencia.
- Puede incluir estado de ejecución, historial, memoria entre sesiones, checkpoints o información necesaria para continuar un proceso.

4	**Human-in-the-Loop**	C11

- Al menos una acción debe poder pausarse antes de ejecutarse.
- El usuario debe poder aprobar, modificar o rechazar la acción propuesta antes de continuar.

5	**Integración externa**	C9 / C13

- Interactuar con al menos un sistema externo: API, base de datos, MCP Server, servicio externo u otra fuente/herramienta.
- La integración debe aportar valor real al caso de uso.

6	**Interfaz accesible**	C14

- La solución debe poder utilizarse sin modificar el código fuente.
- Puede implementarse mediante FastAPI, aplicación web, Streamlit, interfaz conversacional u otra interfaz equivalente.

7	**Ejecución reproducible**	C3 / C14

- Otra persona debe poder descargar el proyecto y ejecutarlo siguiendo instrucciones claras.
- Incluir repositorio, README.md, .env.example, dependencias, instrucciones de instalación/ejecución y Docker/Docker Compose o procedimiento verificable.
- No incluir API Keys, passwords, tokens ni credenciales reales.

8	**Evidencia de calidad**	C15

- Incluir tests de las principales tools y del recorrido principal.
- Incluir un pequeño dataset o conjunto de casos de evaluación y evidencia de trazabilidad.
- Pueden agregarse métricas como task success, tool selection, pasos, latencia, costo y errores.

2. Arquitectura del proyecto

El proyecto puede seguir uno de los siguientes enfoques. La arquitectura seleccionada debe estar justificada en función del problema.

A. Agente único robusto [C9]

- Agente principal
- Tools
- Skill
- Memoria y estado
- HITL
- Integración externa
- Evaluación

B. Sistema multiagente [C12]

- Supervisor u orquestador
- Al menos 2 agentes especializados
- Mecanismo de delegación
- Consolidación de resultados
- Trazabilidad del flujo

CRITERIO No se valorará agregar agentes, frameworks o componentes únicamente para aumentar la complejidad de la solución.

Uso de RAG - C7 / C8

RAG es opcional. Puede utilizarse cuando el proyecto necesite trabajar con documentos, conocimiento propio o información que deba recuperarse dinámicamente. No es necesario incorporarlo si no aporta valor real al caso de uso.

3. Formato de entrega

ENTREGA DIGITAL La entrega se realizará mediante un único repositorio del proyecto. No se considerará entrega final el envío de notebooks sueltos, archivos .py aislados, carpetas comprimidas sin documentación o código enviado por correo.

A. Repositorio

- Entregar la URL del repositorio correspondiente al proyecto (GitHub u otra plataforma equivalente).
- El repositorio debe estar accesible para el equipo docente al momento de la corrección.
- La versión presentada debe coincidir exactamente con la versión utilizada para la evaluación.

Recomendación: crear un tag o release identificable, por ejemplo **v1.0-tfi**

B. README.md obligatorio

El README funcionará como documento principal de la entrega. Debe permitir entender, instalar y probar el proyecto sin necesidad de leer previamente el código.

#	Sección	Contenido esperado
1	Nombre del proyecto	Título claro del proyecto.
2	Integrantes	Nombre y apellido de todos los integrantes.
3	Problema	Qué problema busca resolver.
4	Usuario / destinatario	Quién utilizaría la solución.
5	Descripción	Qué hace el sistema y cuál es el flujo principal.
6	Arquitectura	Diagrama + explicación de LLM, tools, skill, estado/memoria, HITL, integraciones y flujo.
7	Decisiones técnicas	Justificar arquitectura, modelo, RAG/no RAG, persistencia e HITL.
8	Instalación	Paso a paso para instalar el proyecto.
9	Configuración	Variables de entorno necesarias y .env.example sin secretos.
10	Ejecución	Comando exacto para iniciar la aplicación.
11	Uso	Al menos un ejemplo para reproducir el recorrido principal.
12	Evaluación	Qué probaron, con qué casos, qué métricas y qué resultados obtuvieron.
13	Limitaciones	Aspectos fuera de alcance y limitaciones conocidas.

4. Estructura sugerida del repositorio

No es obligatorio utilizar exactamente esta estructura, pero el proyecto debe estar organizado de forma clara.

```
proyecto/
|-- README.md
|-- .env.example
|-- requirements.txt / pyproject.toml
|-- docker-compose.yml
|-- src/ o app/
|   |-- agents/
|   |-- tools/
|   |-- skills/
|   |-- memory/
|   `-- ...
|-- tests/
|-- evals/
|   |-- dataset
|   `-- resultados
`-- docs/
    `-- arquitectura
```

5. Demo

El proyecto deberá poder demostrarse mediante un recorrido principal reproducible. La demo debe permitir observar claramente:

FLUJO DE DEMO Entrada del usuario -> decisión del agente -> uso de tools -> resultado -> intervención humana cuando corresponda -> respuesta final.

No es necesario demostrar todas las funcionalidades. Es preferible mostrar un recorrido principal completo y estable antes que muchas funcionalidades incompletas.

Video de respaldo

- Se recomienda incluir un video breve de 3 a 5 minutos como respaldo.
- Debe mostrar el problema, el sistema funcionando, una decisión agéntica, al menos una tool, HITL y el resultado final.
- El video sirve como respaldo ante problemas de infraestructura, APIs externas o disponibilidad de servicios durante la evaluación.

6. ¿Qué tienen que subir?

En el espacio de entrega deberán indicar únicamente la información necesaria para identificar y acceder al proyecto:

- ☐ Nombre del proyecto
- ☐ Integrantes
- ☐ URL del repositorio
- ☐ URL de la demo desplegada, si existe
- ☐ URL del video de demo, si se realizó
- ☐ Breve descripción del proyecto (máximo 5 líneas)

IMPORTANTE No es necesario adjuntar nuevamente todo el código si ya se encuentra correctamente disponible en el repositorio.

7. Entregas parciales sugeridas

Las entregas parciales permiten recibir feedback progresivamente y reducir el riesgo de llegar a la Clase 16 con problemas importantes de alcance o arquitectura.

PARCIAL 1 - después de C10	PARCIAL 2 - después de C14
• Nombre e integrantes • Problema y usuario • Caso de uso y alcance • Solución propuesta • Arquitectura preliminar • Tools previstas • Skill propuesta	• Agente funcionando • Tools principales • Integración externa • Estado o memoria • HITL • API o interfaz • Demo del flujo principal

Objetivo al llegar a la Clase 16: corregir -> evaluar -> documentar -> preparar la demo -> preparar la defensa.

8. Entrega final y criterios de evaluación

FECHA LÍMITE Domingo 8 de noviembre de 2026 - 23:59 h

A partir de ese momento se considerará como entrega la versión identificada en el repositorio.

La entrega final deberá incluir

- ☐ Proyecto funcional
- ☐ Repositorio completo
- ☐ README.md
- ☐ Archivo .env.example
- ☐ Instrucciones de instalación y ejecución
- ☐ Tests
- ☐ Dataset o casos de evaluación
- ☐ Resultados de evaluación
- ☐ Diagrama de arquitectura
- ☐ Documentación de decisiones técnicas
- ☐ Demo reproducible

Rúbrica

Criterio	Peso
Funcionamiento end-to-end	30%
Diseño agéntico	25%
Evaluación y trazabilidad	20%
Arquitectura y decisiones técnicas	15%
Presentación y documentación	10%

9. Checklist antes de entregar

- ☐ El repositorio es accesible para el equipo docente.
- ☐ La versión entregada está identificada (tag/release recomendado).
- ☐ El README permite entender, instalar y ejecutar el proyecto.
- ☐ Existe un .env.example y no hay credenciales reales versionadas.
- ☐ El proyecto tiene al menos 2 tools y una decisión agéntica real.
- ☐ Existe al menos una skill reutilizable.
- ☐ Estado/memoria y persistencia básica funcionan.
- ☐ Existe al menos una instancia de HITL.
- ☐ Existe al menos una integración externa.
- ☐ La aplicación tiene API o interfaz accesible.
- ☐ Los tests principales corren y existe evidencia de evaluación.
- ☐ El recorrido principal de la demo es reproducible.
- ☐ El diagrama de arquitectura coincide con la implementación.
- ☐ Las decisiones técnicas principales están justificadas.
- ☐ Los links de repositorio, demo y video fueron cargados correctamente.

REGLA SIMPLE Al momento de entregar deberíamos poder responder claramente: ¿Qué problema resuelve? -> ¿Cómo funciona? -> ¿Por qué está diseñado así? -> ¿Funciona realmente? -> ¿Cómo lo demostraron?